KTÜ OF TEKNOLOJI FAKÜLTESI · YAZILIM MÜHENDISLIĞI BÖLÜMÜ

Nesne Yönelimli Programlama

2026–2027 Güz Dönemi · Laboratuvar Föyü

HAFTA 06

Laboratuvar Föyü

Statik sayaç ve otomatik numara · yardımcı sınıf · static final sabitler ve savunmalı kopya · static/örnek hata ayıklama · tekil kayıtçı

ÖZET

Bu laboratuvar oturumunda önce demirbaş numarasını **sınıfın kendisinin** ürettiği bir `Kitap` sınıfı yazacak, ardından üç kurala tam olarak uyan bir **yardımcı (utility) sınıf** tasarlayacaksınız. Üçüncü görevde ders değerlendirme kurallarını `static final` sabitlerde toplayacak, `static` blokla kurulumu doğrulayacak ve sabit diziyi **savunmalı kopya** ile koruyacaksınız. Dördüncü görevde `static` ve örnek ayrımı yanlış kurulmuş bir sınıfı hata ayıklayarak düzeltecek, bonus görevde ise sistemde tek bir örneği bulunan bir **kayıtçı (logger)** yazacaksınız.

Ders	Nesne Yönelimli Programlama (Zorunlu)
Program / Sınıf	Yazılım Mühendisliği · 2. Sınıf · Güz
Haftalık Yük	2 saat teori + 2 saat laboratuvar (toplam 4 saat)
Öğretim Elemanı	Dr. Öğr. Üyesi Rıfat BENVENİSTE
Laboratuvar Sorumlusu	Arş. Gör. Betül MUMCU
Programlama Dili	Java (JDK 21 LTS)
Geliştirme Ortamı	Visual Studio Code + Extension Pack for Java

Oturum Bilgileri

	Bilgi
Süre	120 dakika (2 ders saati)
Sorumlu	Arş. Gör. Betül MUMCU
Ortam	Visual Studio Code + JDK 21 (LTS)
Çalışma biçimi	Bireysel
Ön koşul	Hafta 6 ders notu Bölüm 2, 3 ve 4 okunmuş olmalıdır
Teslim	Tüm `.java` dosyaları tek bir sıkıştırılmış klasör hâlinde, oturum bitiminde
Dosya adı biçimi	`Hafta06_<Numara>_<AdSoyad>.zip`
Değerlendirme	Görev 1–4 zorunlu (80 puan) · Görev 5 ileri düzey (20 puan)

Bu Bölümün Sonunda Öğrenci

- `static` alan ile örnek alanı doğru ayırt eder; bir üyeyi hangi düzeye koyacağına **gerekçeli** karar verir.
- Otomatik ve benzersiz kimlik üreten **sayaç desenini** uygular; benzersizlik güvencesini çağıran koddan sınıfa taşır.
- Üç kurala uyan bir **yardımcı sınıf** yazar: tüm üyeler `static`, yapıcı `private`, sınıf `final`.
- `static final` sabitlerle sihirli sayıları ortadan kaldırır; `static` blokla kurulum tutarlılığını doğrular.
- `final`'ın referans türlerde neyi korumadığını gösterir ve **savunmalı kopya** yazar.
- `static` / örnek ayrımı yanlış kurulmuş bir sınıfı çalışan çıktı üzerinden teşhis eder ve düzeltir.
- `private` yapıcı ile `static` alanı birleştirerek tek örnekli bir sınıf (tekil sezgisi) yazar.



Şekil 6.1 — Laboratuvar oturumunun zaman planı

Çalışma Kuralları

1. Her `public` sınıf için **ayrı bir `.java` dosyası** oluşturun. Dosya adı, içindeki `public` sınıfın adıyla birebir aynı olmalıdır.

2. Kaynak dosyaları `src`, derlenmiş dosyaları `bin` klasöründe tutun: `javac -d bin src/*.java`
3. Tanımlayıcılarda (sınıf, metot, değişken adları) **Türkçe karakter kullanmayın**. Ekrana yazdırılan metinlerde de ASCII kalın.
4. Alanları **üç grup hâlinde** bildirin: önce `static final` sabitler, sonra `static` alanlar, en sonra örnek alanları. Her grubun başına bir yorum satırı koyun.
5. `static` alanları `private` tutun; dışarıya `static` bir getter ile açın. Doğrudan yazılabilir `public static` alan bırakmayın.
6. `static` üyelere **sınıf adı üzerinden** erişin: `Kitap.uretilenSayisi()`. `nesne.staticUye()` yazımı puan kaybettirir.
7. Görevleri sırayla yapın; her görev bir öncekinin kavramını kullanır.

> **İNTİHAL UYARISI**
>
> Teslim edilen kod **bireysel** çalışmanın ürünü olmalıdır. Birbirinin kopyası olduğu belirlenen teslimlerin tamamı değerlendirme dışı bırakılır. Arkadaşınızla tartışabilirsiniz; kod alışverişi yapamazsınız.

Görev 0 — Ortam ve Ön Koşul Doğrulaması

Süre	Puan	Zorluk
10 dakika	Puansız — **zorunlu**	●○○

Amaç

Java ortamının çalıştığını doğrulamak ve `static` alan ile örnek alanın farkını en küçük ölçekte gözlemlemek.

Görev Tanımı

`Gorev0_Kontrol` adında bir sınıf yazın. Sınıfın bir `static` bir de örnek sayacı olsun; yapıcı her ikisini de artırsın. Üç nesne üretip iki sayacın değerlerini yazdırın.

Gereksinimler

1. `static int sinifSayaci` ve `int nesneSayaci` alanları tanımlanmalıdır.
2. Yapıcı her iki sayacı da **birer artırmalıdır**.
3. `main` Java sürümünü ve işletim sistemini yazdırmalıdır.
4. Üç nesne üretilmeli ve her birinin `nesneSayaci` değeri ayrı ayrı yazdırılmalıdır.
5. Program sonunda `sinifSayaci == 3` ve `nesneSayaci == 1` koşulu sınanıp bir sonuç iletisi yazdırılmalıdır.

Beklenen Çıktı

```
▸ TERMINAL — java Gorev0_Kontrol
=== Hafta 06 - Gorev 0 : Ortam Kontrolu ===
  Java surumu : 25.0.2
  Isletim sis.: Linux

--- static / ornek ayrimi mini denemesi ---
  sinifSayaci (static) = 3
  a.nesneSayaci        = 1
  b.nesneSayaci        = 1
  c.nesneSayaci        = 1

  SONUC: ORTAM HAZIR. Gorev 1'e gecebilirsiniz.
```

> **İPUCU**
>
> Java sürümünü `System.getProperty("java.version")`, işletim sistemini `System.getProperty("os.name")` ile okuyabilirsiniz. Sürüm numaranız farklı olabilir; bu bir sorun değildir. Asıl beklenen gözlem, üç nesnenin

> `nesneSayaci` değerinin **1** kalmasına karşın `sinifSayaci` değerinin **3** olmasıdır.

Kontrol Soruları

1. Yapıcı her iki sayacı da bir artırdığı hâlde biri 3, diğeri 1 oldu. Bunun nedeni nedir?
2. `nesneSayaci` alanının önüne `static` yazsaydınız çıktı nasıl değişirdi? Yanıtınızı çalıştırmadan önce yazınız.

Görev 1 — Statik Sayaç ve Otomatik Numara Üreteci

Süre	Puan	Zorluk
30 dakika	20 puan	●○○

Amaç

Benzersiz kimlik üretme sorumluluğunu çağıran koddan alıp **sınıfın kendisine** vermek; sınıf geneli ölçümleri `static` alanlarda tutmak.

Görev Tanımı

Bir kütüphane için `Kitap` sınıfı yazın. Her kitap üretildiğinde demirbaş numarası **otomatik** verilsin; çağıran kod numarayla hiç ilgilenmesin. Ayrıca `Gorev1_KitapDemo` adında bir sürücü program yazarak davranışı gösterin.

Gereksinimler

Üye	Düzey	Beklenen davranış
`ON_EK`, `BASLANGIC_NO`	`private static final`	`"KTP-"` ve `1` — koda gömülü değer olmayacak
`AZAMI_ODUNC_GUN`	`public static final`	`21` — dışarıdan okunabilir kural
`sonrakiNo`, `uretilen`, `oduncteki`	`private static`	Sınıf geneli sayaçlar; dışarıdan yazılamaz.
`demirbasNo`, `baslik`, `yazar`	`private final` (örnek)	Her kitapta farklı; bir kez atanır
`odunctemi`	`private` (örnek)	Kitabın kendi durumu
`Kitap(String baslik, String yazar)`	yapıcı	Numarayı **istemez**, `numaraUret()` ile alır
`numaraUret()`	`private static`	`KTP-00001` biçimi; `String.format("%s%05d", ...)` kullanın

`oduncVer()`, `iadeAl()`	örnek metodu	Kitabın durumunu değiştirir; `oduncteki` sayacını günceller
`uretilenSayisi()`, `oduncteSayisi()`, `raftakiSayisi()`	`static`	Sınıf geneli sorulara yanıt verir
`oduncOrani()`	`static`	Yüzde; hiç kitap yokken **sıfıra bölme olmamalı**
`sonrakiNumara()`	`static`	Sıradaki numarayı üretmeden gösterir
Geçersiz değer	—	Boş başlık/yazar `IllegalArgumentException` ile reddedilir; **sayaç artmaz**

Beklenen Çıktı

```
▸ TERMINAL — java Gorev1_KitapDemo (ilk bölüm)
=== 1) Hic kitap yokken sinif durumu ===
  uretilenSayisi = 0
  sonrakiNumara  = KTP-00001

=== 2) Bes kitap uretiliyor ===
  KTP-00001 | Nesne Yonelimli Programlama / Deitel | RAFTA
  KTP-00002 | Core Java Volume I / Horstmann | RAFTA
  KTP-00003 | Effective Java / Bloch | RAFTA
  KTP-00004 | Java: The Complete Reference / Schildt | RAFTA
  KTP-00005 | Clean Code / Martin | RAFTA
  Numaralar KESINTISIZ ve BENZERSIZ verildi.

=== 3) Odunc islemleri ===
  UYARI: KTP-00004 zaten odunc verilmis.
  oduncteSayisi = 2
  raftakiSayisi = 3
  oduncOrani    = 40.0%
```

```
▸ TERMINAL — java Gorev1_KitapDemo (ikinci bölüm)
=== 4) Iade ===
  UYARI: KTP-00001 zaten raflarda.
  oduncteSayisi = 1
  oduncOrani    = 20.0%

=== 5) Sayaclar nesneden BAGIMSIZ okunabilir ===
  Kitap.uretilenSayisi() = 5
  k3.uretilenSayisi()    = 5   <- ayni deger, ama bu yazim bicimi ONERILMEZ
  Kitap.sonrakiNumara()  = KTP-00006

=== 6) Gecersiz kitap sayaci artirmaz ===
  RET : Baslik bos olamaz.
  uretilenSayisi = 5   sonrakiNumara = KTP-00006
```

DİKKAT

Son bölüm bu görevin **en kritik** ölçütüdür: geçersiz bir kitap reddedildiğinde `uretilenSayisi` ve `sonrakiNumara` değerleri **değişmemelidir**. Bunu sağlamak için doğrulamayı `numaraUret()` çağrısından **önce** yapmalısınız. Sırayı ters kuran öğrenci, hiç var olmamış bir kitap için numara harcamış olur.

İPUCU

`String.format("%s%05d", ON_EK, sonrakiNo)` ifadesi `KTP-00001` üretir; `%05d` sayıyı beş basamağa sıfırla tamamlar. `oduncOrani()` metodunda `uretilen == 0` denetimini unutmayın; aksi hâlde hiç kitap yokken `ArithmeticException` ya da `NaN` alırsınız.

Kontrol Soruları

1. `sonrakiNo` alanını `static` yapmasaydınız ne olurdu? Üretilen kitapların numaraları ne olurdu?
2. `numaraUret()` metodunu `static` yapmak zorunda mıydınız? Teknik olarak zorunlu olmasa bile neden doğru tercih olduğunu açıklayınız.
3. Bir kitap nesnesi çöp toplayıcı tarafından yok edilse `uretilen` sayacı azalır mı? Bu davranış doğru mudur, gerekçelendiriniz.

Görev 2 — Yardımcı (Utility) Sınıf

Süre	Puan	Zorluk
30 dakika	20 puan	●●○

Amaç

Durumu olmayan, yalnızca hesap yapan bir sınıfın üç kurala uygun biçimde nasıl yazıldığını uygulamak.

Görev Tanımı

`BirimCevirici` adında bir yardımcı sınıf yazın. Sınıfın **hiçbir örnek üyesi bulunmasın**. `Gorev2_CeviriciDemo` adında bir sürücü program yazarak tüm dönüşümleri ve ret durumlarını gösterin.

Gereksinimler

Ölçüt	Beklenen
Sınıf bildirimi	`public final class BirimCevirici`
Yapıcı	`private`; gövdesinde `throw new AssertionError(...)`
Örnek üyesi	**Hiç olmayacak** — tek bir tane bile
Dönüşüm sabitleri	`KM_MIL`, `KG_LIBRE`, `LITRE_GALON`, `MUTLAK_SIFIR_C` — `public static final double`
Uzunluk / kütle / hacim	`kmToMil`, `milToKm`, `kgToLibre`, `litreToGalon`
Sıcaklık	`celsiusToFahrenheit`, `celsiusToKelvin`
Yardımcı	`yuvarla(double deger, int basamak)` — 0–8 basamak
Çağrı sayacı	`private static int cagriSayisi` + `static` getter
Doğrulama	Negatif mesafe/kütle/hacim ve mutlak sıfır altı sıcaklık `IllegalArgumentException` ile reddedilir
Sayaç kuralı	Sayaç **yalnızca başarılı** dönüşümlerde artar — doğrulama sayaçtan önce çalışır

Beklenen Çıktı

```
▸ TERMINAL — java Gorev2_CeviriciDemo (dönüşümler)
=== 1) Nesne URETMEDEN dogrudan cagri ===
  100 km   = 62.137 mil
  26.2 mil = 42.165 km
  75 kg    = 165.347 libre
  50 litre = 13.209 galon
```

```
=== 2) Sicaklik donusum tablosu ===
   Celsius Fahrenheit     Kelvin
     -40.0      -40.0     233.15
       0.0       32.0     273.15
      25.0       77.0     298.15
      37.0       98.6     310.15
     100.0      212.0     373.15

=== 3) Sabitlere dogrudan erisim ===
  KM_MIL         = 0.621371
  KG_LIBRE       = 2.204623
  MUTLAK_SIFIR_C = -273.15
```

```
▸ TERMINAL — java Gorev2_CeviriciDemo (ret ve sayaç)
=== 4) Gecersiz girdiler reddediliyor ===
  RET : Mesafe negatif olamaz: -5.0
  RET : Mutlak sifirin altinda sicaklik olamaz: -300.0
  RET : Basamak 0-8 araliginda olmalidir.

=== 5) Static sayac: kac basarili donusum yapildi ===
  cagriSayisi = 14
  Ret edilen cagrilar sayaca ISLENMEDI: dogrulama
  say() cagrisindan ONCE yapilmaktadir.

=== 6) Sinifin nesnesi uretilemez ===
  new BirimCevirici();  // yazilirsa derleme hatasi:
  The constructor BirimCevirici() is not visible
```

İPUCU

Sıcaklık tablosunu `System.out.printf("  %8.1f %10.1f %10.2f%n", ...)` ile hizalayabilirsiniz. `yuvarla()` metodunda `Math.round(deger * carpan) / carpan` deseni kullanın; `carpan` değerini `Math.pow(10, basamak)` ile üretin.

DİKKAT

Sayacı artıran `say()` çağrısını doğrulama denetimlerinden **sonra** yazınız. Ret edilen bir çağrı sayaca işlenmemelidir. Beklenen çıktıda `cagriSayisi = 14` görülür; üç ret edilen çağrı bu sayıya dahil değildir.

Kontrol Soruları

1. Yapıcıyı `private` yapmasaydınız derleyici ne yapardı? Bu neden bir sorun olurdu?
2. Sınıfı `final` yapmanın gerekçesi nedir? `private` yapıcılı bir sınıfın alt sınıfı zaten yazılabilir mi?
3. `cagriSayisi` alanı bir örnek alanı olsaydı ne olurdu? Bu sınıf bağlamında sorunun anlamlı olup olmadığını tartışınız.

Görev 3 — `static final` Sabitler ve Savunmalı Kopya

Süre	Puan	Zorluk
25 dakika	20 puan	●●○

Amaç

Sihirli sayıları adlandırılmış sabitlere dönüştürmek, `static` blokla kurulum tutarlılığını doğrulamak ve dizi sabitlerini savunmalı kopyayla korumak.

Görev Tanımı

`SinavKurallari` adında bir sınıf yazın. Sınıf, `YZM205` dersinin değerlendirme kurallarını barındırsın. `Gorev3_KurallarDemo` adında bir sürücü programla dört öğrenciyi değerlendirin ve savunmalı kopyanın işe yaradığını gösterin.

Gereksinimler

Üye	Düzey	Beklenen
`VIZE/ODEV/LAB/FINAL_AGIRLIGI`	`public static final double`	`0.30 / 0.10 / 0.20 / 0.40`
`ASGARI_FINAL_NOTU`, `ASGARI_DEVAM_YUZDE`, `GECME_NOTU`	`public static final int`	`40`, `70`, `50`
`DERS_KODU`	`public static final String`	`"YZM205"`
`ESIKLER`, `HARFLER`	`private static final` dizi	`{50,60,70,80,90}` ve `{FF,DD,CC,BB,BA,AA}`
`static` blok	—	Ağırlıklar toplamı 1.0 değilse `ExceptionInInitializerError` fırlatılır
`esikler()`	`public static int[]`	**Savunmalı kopya** — `ESIKLER.clone()` döndürür
`basariNotu(vize, odev, lab, son)`	`public static double`	Ağırlıklı ortalama; her not 0–100 aralığında denetlenir
`harfNotu(basari)`	`public static String`	Eşiklere göre harf
`gectiMi(basari, final, devam)`	`public static boolean`	Üç koşulun tamamı sağlanmalı
`gerekce(...)`	`public static String`	Kalma nedenini metin olarak açıklar
Yapıcı	`private`	Sınıf `final`, örnek üretilemez

Beklenen Çıktı

```
▸ TERMINAL — java Gorev3_KurallarDemo (kurulum ve değerlendirme)
=== 1) Derleme zamani sabiti sinifi YUKLEMEZ ===
  DERS_KODU = YZM205
  static blok CIKTISI HENUZ YOK: DERS_KODU bir
  static final String'tir ve degeri derleme aninda
  cagiran sinifin icine GOMULMUSTUR.

=== 2) Ilk METOT cagrisi: sinif SIMDI yuklenir ===
  SinavKurallari.esikler() cagriliyor...
  [static blok] YZM205 kurallari dogrulandi (agirlik toplami=1.0)
  -> 5 esik okundu

=== 3) Kural sabitleri ===
  vize/odev/lab/final = 0.3 / 0.1 / 0.2 / 0.4
  ASGARI_FINAL_NOTU   = 40
  ASGARI_DEVAM_YUZDE  = 70
  GECME_NOTU          = 50
```

```
▸ TERMINAL — java Gorev3_KurallarDemo (savunmalı kopya ve ret)

=== 4) Dort ogrenci degerlendiriliyor ===
  Ad        Basari  Harf Durum
  Elif       80.00    BA GECTI
  Mert       49.50    FF FINAL BARAJI (final=35.0 < 40)
  Zeynep     91.20    AA GECTI
  Kaan       57.50    DD DEVAMSIZ (devam=55% < 70%)

=== 5) Savunmali kopya: sabit disaridan bozulamaz ===
  esikler()  : 50 60 70 80 90
  kopya[0] = 5 yapildi.
  esikler()  : 50 60 70 80 90     <- ASIL dizi bozulmadi
  harfNotu(47) = FF

=== 6) Gecersiz not reddediliyor ===
  RET : final notu 0-100 araliginda olmalidir: 130.0
```

GÖZLEMLENECEK AYRINTI — derleme zamanı sabiti

Çıktının ilk bölümüne dikkat ediniz: `DERS_KODU` sabitine erişmek sınıfı **yüklemedi**; `static` blok çıktısı ancak ilk metot çağrısında göründü. Bunun nedeni `static final String` sabitinin değerinin derleme anında çağıran sınıfın içine gömülmesidir (ders notu Bölüm 4.2). Sürücü programınızda bu ayrımı görünür kılmanız beklenmektedir.

İPUCU

Savunmalı kopyayı sınamak için: `esikler()` ile bir kopya alın, kopyanın `0.` elemanını `5` yapın, sonra `esikler()` metodunu **yeniden** çağırıp asıl dizinin bozulmadığını gösterin. `harfNotu(47)` çağrısının hâlâ `FF` döndürmesi kanıttır.

Kontrol Soruları

1. `ESIKLER` dizisini `public static final` yapsaydınız hangi saldırı mümkün olurdu? Üç satırlık bir örnekle gösteriniz.
2. `static` bloğun içindeki denetim başarısız olursa program hangi aşamada durur? Bu, dönem boyunca yanlış not hesaplamaya göre neden daha iyidir?
3. `clone()` çağrısı bu dizide yeterli olmuştur. Hangi durumda yetmezdi?

Görev 4 — `static` / Örnek Karışık Senaryoda Hata Ayıklama

Süre	Puan	Zorluk
25 dakika	20 puan	●●○

Amaç

Derleyicinin **sessiz kaldığı** bir `static` hatasını çalışan çıktı üzerinden teşhis etmek ve doğru düzeyi gerekçelendirerek düzeltmek.

Görev Tanımı

Aşağıdaki `BozukTakim` sınıfı size verilmiştir. Sınıf hatasız derlenir ve çalışır; ancak yanlış sonuç üretir. Görev üç aşamalıdır: (i) sınıfı olduğu gibi çalıştırıp hatayı **çıktıda gösterin**, (ii) hangi alanların yanlış düzeyde olduğunu tespit edin, (iii) `DuzgunTakim` adında düzeltilmiş bir sürüm yazın.

BozukTakim.java (bozuk sürüm)

```
1  // BOZUK SURUM - Gorev 4te bunu duzelteceksiniz
2  class BozukTakim {
3      static String ad;                          // (?)
4      static int puan = 0;                       // (?)
5      static String[] oyuncular = new String[3]; // (?)
6      static int oyuncuSayisi = 0;               // (?)
7      static int kurulanTakim = 0;               // (?)
8
9      BozukTakim(String ad) {
10         BozukTakim.ad = ad;
11         kurulanTakim++;
12     }
13
14     void golAt() { puan += 3; }
15
16     void oyuncuEkle(String o) {
17         if (oyuncuSayisi < oyuncular.length) {
18             oyuncular[oyuncuSayisi] = o;
19             oyuncuSayisi++;
20         }
```

```
    }

    public String toString() {
        return ad + " puan=" + puan + " oyuncu=" + oyuncuSayisi;
    }
}
```

Kod 6.1 — Düzeltilecek sınıf (verilen hâli)

Gereksinimler

Aşama	Beklenen
(i) Hatayı gösterme	Üç takım kurun, ikisine gol attırın, birine oyuncu ekleyin; üç takımın da aynı adı ve puanı gösterdiğini yazdırın
(ii) Teşhis	Beş alanın her biri için `static` mi örnek mi olması gerektiğini **gerekçesiyle** yorum satırına yazın
(iii) `DuzgunTakim`	`ad`, `puan`, `oyuncular`, `oyuncuSayisi` → örnek alanı (`private`)
	`kurulanTakim`, `toplamPuan` → `private static` (lig geneli ölçüm)
	`GALIBIYET_PUANI` → `public static final int` = `3`
Doğrulama	Boş takım adı `IllegalArgumentException` ile reddedilir
Kadro sınırı	`oyuncular` dizisi 3 kişiliktir; dolu kadroya ekleme uyarı yazar, istisna fırlatmaz
`static` metotlar	`kurulanTakim()`, `toplamPuan()`, `ortalamaPuan()` — sıfıra bölme korumalı
Karşılaştırma	Aynı senaryoyu her iki sınıfla çalıştırıp çıktıları yan yana gösterin

Beklenen Çıktı

```
▸ TERMINAL — java Gorev4_HataAyiklama (bozuk ve düzgün sürüm)
=== 1) BOZUK surum: uc takim kuruluyor ===
  b1 -> Ege puan=6 oyuncu=2
  b2 -> Ege puan=6 oyuncu=2
  b3 -> Ege puan=6 oyuncu=2
  Uc takim da AYNI adi, AYNI puani, AYNI kadroyu
  gosteriyor: ad/puan/oyuncular yanlislikla static.

=== 2) DUZGUN surum: ayni senaryo ===
  d1 -> Karadeniz puan=3 oyuncu=1
  d2 -> Akdeniz puan=3 oyuncu=1
  d3 -> Ege puan=0 oyuncu=0

=== 3) Static KALMASI gereken olcumler ===
  kurulanTakim = 3
  toplamPuan   = 6
  ortalamaPuan = 2.00
```

```
▸ TERMINAL — java Gorev4_HataAyiklama (kadro sınırı ve karar kuralı)

=== 4) Kadro sinirinin ornek olmasi ===
  UYARI: Karadeniz kadrosu dolu.
  d1 -> Karadeniz puan=3 oyuncu=3
  d2 -> Akdeniz puan=3 oyuncu=1   (etkilenmedi)

=== 5) Karar kurali ===
  ad, puan, oyuncular : nesneye ozgu -> ORNEK
  kurulanTakim, toplamPuan : lig geneli -> STATIC
  GALIBIYET_PUANI : degismeyen kural -> STATIC FINAL
```

DİKKAT

Bu görevin en sık yapılan hatası, düzeltme sırasında **tüm** `static` alanları kaldırmaktır. `kurulanTakim` ve `toplamPuan` alanları `static` **kalmalıdır**; çünkü bunlar gerçekten bir takıma değil, lige aittir. Doğru düzeltme, `static`'i silmek değil, her alanı **doğru düzeye** taşımaktır.

İPUCU

Her alan için şu iki soruyu sırayla sorun: (1) “Bu değer tüm takımlar için aynı mı?” (2) “Aynıysa hiç değişiyor mu?” İlk soruya hayır derseniz örnek alanı, evet–hayır derseniz `static`, evet–evet derseniz `static final` seçin. Ders notu Bölüm 4.7'deki karar akışı bu iki soruyu şema hâline getirir.

Kontrol Soruları

1. Bozuk sürümde üç takım da neden `Ege` adını gösterdi? Hangi satır buna yol açtı?
2. `kurulanTakim` alanı bozuk sürümde de `static`'ti ve bu **doğruydu**. Diğer dört alandan farkı nedir?
3. Bu hatanın derleme aşamasında yakalanmamasının nedeni nedir? Derleyici hangi bilgiye sahip değildir?

Görev 5 — İleri Düzey: Tekil Kayıtçı (Bonus)

Süre	Puan	Zorluk
20 dakika	20 puan (bonus)	●●●

Amaç

Hafta 5'te öğrenilen `private` yapıcıyla bu haftanın `static` alanını birleştirerek, sistemde yalnızca tek örneği bulunan bir sınıf yazmak; erken ve tembel başlatma arasındaki farkı gözlemlemek.

Görev Tanımı

`Kayitci` adında, sistemde tek örneği bulunan bir günlük (log) sınıfı yazın. İki farklı servis metodu aynı kayıtçıyı kullansın. Ayrıca `TembelSayac` adında, nesnesi **ilk istendiğinde** üretilen ikinci bir sınıf yazıp iki başlatma biçimini karşılaştırın.

Gereksinimler

Üye	Beklenen
`SEVIYE_HATA/UYARI/BILGI`	`public static final int` = `0/1/2`
`SEVIYE_ADI`	`private static final String[]` — ekran etiketleri
`uretimSayisi`	`private static int`; bildirimi `ORNEK`'ten **önce** olmalı
`ORNEK`	`private static final Kayitci` — erken (eager) başlatma
Yapıcı	`private`; çağrıldığını ekrana yazar
`getInstance()`	`public static Kayitci` — tek kapı
`esikSeviye`, `yazilan`, `elenen`	örnek alanları — kayıtçının kendi durumu
`yaz(int seviye, String ileti)`	Seviye eşiği aşarsa eler, yoksa yazar
`setEsikSeviye(int)`	Geçersiz seviye uyarı yazar, değişiklik yapmaz
`TembelSayac`	`private static TembelSayac ornek;` — `getInstance()` içinde `if (ornek == null)` ile üretilir
Gösterilecek	`k1 == k2` → `true`; `uretimSayisi` → `1`; esik değişince iki servisin de etkilenmesi

Beklenen Çıktı

```
▸ TERMINAL — java Gorev5_Kayitci (tek örnek ve eşik)
=== 1) main basladi ===

=== 2) Ilk getInstance : sinif yuklenir ===
  [Kayitci yapici] cagri sayisi = 1
  k1 == k2 ? true
  uretimSayisi = 1

=== 3) Iki farkli servis AYNI kayitciyi kullaniyor ===
    [BILGI] servisA basladi
    [HATA ] servisA: veri kaynagi bulunamadi
    [UYARI] servisB: onbellek bos
    [BILGI] servisB bitti
  yazilan = 4   elenen = 0

=== 4) Esik degisince TUM servisler etkilenir ===
  esik SEVIYE_HATA yapildi; yeniden calistiriliyor:
    [HATA ] servisA: veri kaynagi bulunamadi
  yazilan = 5   elenen = 3
  Tek nesne oldugu icin ayar TEK yerden yonetilir;
```

```
  ancak bu ayni zamanda deseni kirilgan yapan yandir.
```

```
▸ TERMINAL — java Gorev5_Kayitci (tembel başlatma)
=== 5) Gecersiz seviye reddediliyor ===
  UYARI: gecersiz seviye, degisiklik yapilmadi.

=== 6) Tembel baslatma: nesne HENUZ uretilmedi ===
  TembelSayac.uretimSayisi() = 0
  [TembelSayac yapici] ilk istekte uretildi
  t1 == t2 ? true
  t2.deger() = 3   (t1'den artirilanlar dahil)
  uretimSayisi = 1

=== 7) Hafta 14 notu ===
  Tembel surum tek is parcacigi icin dogrudur; cok
  is parcacikli ortamda iki nesne uretebilir. Guvenli
  bicimler Hafta 14'te ele alinacaktir.
```

TUZAK — bildirim sırası

`uretimSayisi` alanının bildirimi `ORNEK` bildiriminden **önce** yazılmalıdır. Sıra ters olursa yapıcı sayacı `1` yapar, hemen ardından `= 0` başlatıcısı onu sıfırlar ve çıktıda `uretimSayisi = 0` görürsünüz. Bu, ders notu Bölüm 4.1'deki "`static` alan başlatıcıları yazım sırasını izler" kuralının doğrudan sonucudur. Yanlış sırayı bilerek deneyip çıktıyı raporunuza ekleyiniz.

İPUCU

Servis metotlarını `static void servisA()` ve `static void servisB()` biçiminde yazıp her ikisinin de `Kayitci.getInstance()` çağırmasını sağlayın. Eşiği `SEVIYE_HATA` yaptıktan sonra iki servisi yeniden çalıştırın; `elenen` sayacının arttığını gösterin — bu, tek nesnenin paylaşıldığının kanıtıdır.

Kontrol Soruları

1. `k1 == k2` neden `true` döndürdü? `equals` ezilmemiş olmasına rağmen bu sonuç neden beklenendir?
2. Erken (eager) ve tembel (lazy) başlatmanın her birinin bir üstünlüğünü ve bir sakıncasını yazınız.
3. Tek nesne olmasının yarattığı **risk** nedir? Bir ayarın tek yerden yönetilmesi neden aynı zamanda bir sorundur?

Değerlendirme ve Teslim

Puanlama Ölçütleri

Görev	Ölçüt	Puan
Görev 1	Sayaçlar `private static` (5) · numara sınıfça üretiliyor (6) · sabitler tanımlı (3) · geçersiz kitap sayacı artırmıyor (6)	20
Görev 2	Üç kural (yapıcı `private`, sınıf `final`, tüm üyeler `static`) (8) · dönüşümler doğru (5) · doğrulama sayaçtan önce (4) · sabitler `static final` (3)	20
Görev 3	Sabitler eksiksiz (5) · `static` blok doğrulaması (6) · savunmalı kopya (6) · not denetimi (3)	20
Görev 4	Hata çıktıda gösterilmiş (5) · beş alanın teşhisi gerekçeli (6) · doğru düzeye taşınmış (6) · `static` kalması gerekenler korunmuş (3)	20
Görev 5	Üç parça (`ORNEK`, `private` yapıcı, `getInstance`) (8) · bildirim sırası doğru (4) · eşik paylaşımı gösterilmiş (4) · tembel/erken karşılaştırması (4)	20 (bonus)
Toplam	Görev 1–4 zorunlu · Görev 5 bonus	**80 + 20**

Aşağıdaki durumlarda görev puanından kesinti uygulanır:

Durum	Kesinti
Nesneye özgü bir alan `static` bırakılmış	Görev puanının %50'si
`static` alan `public` bırakılmış (getter yerine doğrudan erişim)	Görev puanının %30'u
Dizi sabiti `public static final` verilmiş (savunmalı kopya yok)	Görev puanının %40'ı
Yardımcı sınıfın yapıcısı `public` bırakılmış	Görev puanının %40'ı
`nesne.staticUye()` biçiminde çağrı	3 puan
Alanlar üç grup hâlinde bildirilmemiş / yorum yok	3 puan
Sıfıra bölme koruması yok (`oduncOrani`, `ortalamaPuan`)	4 puan
Türkçe karakter içeren tanımlayıcı	3 puan
Dosya adı ile sınıf adı uyuşmuyor / derlenmiyor	Görev puanının tamamı

Teslim Öncesi Kontrol Listesi

- [ ] Tüm dosyalar hatasız derleniyor (`javac -d bin src/*.java` uyarı vermiyor).
- [ ] Her sınıfta alanlar üç grup hâlinde ve yorumlu olarak bildirildi.
- [ ] Nesneye özgü hiçbir alan `static` değil; sınıf geneli hiçbir ölçüm örnek alanı değil.
- [ ] Tüm `static` alanlar `private`; dışarıya `static` getter ile açılıyor.
- [ ] Dizi sabitlerim `private static final` ve `clone()` ile kopya döndürülüyor.
- [ ] `BirimCevirici` sınıfımın hiçbir örnek üyesi yok; yapıcısı `private`, sınıf `final`.
- [ ] `static` üyelere her yerde sınıf adı üzerinden erişiyorum (`Kitap.uretilenSayisi()`).
- [ ] Sıfıra bölme korumalarını yazdım (`oduncOrani`, `ortalamaPuan`).
- [ ] Geçersiz girdiler reddedildiğinde sayaçlar artmıyor.
- [ ] Dosya adları içindeki `public` sınıf adlarıyla birebir aynı; arşiv adı `Hafta06_<Numara>_<AdSoyad>.zip` biçiminde.

Sık Karşılaşılan Hatalar

Belirti	Olası neden	Çözüm
`Cannot make a static reference to the non-static field x`	`main` ya da `static` metottan örnek alanına erişiliyor	Nesne üretin; alanı gelişigüzel `static` yapmayın
`Cannot use this in a static context`	`static` metotta `this` yazılmış	`this`'i kaldırın
Tüm nesneler aynı değeri gösteriyor	Nesneye özgü alan `static` yapılmış	Alanı örnek düzeyine taşıyın
`The constructor BirimCevirici() is not visible`	Yardımcı sınıfın nesnesi üretilmeye çalışılıyor	Nesne üretmeyin; `static` metotları doğrudan çağırın
`ExceptionInInitializerError`	`static` blok istisna fırlattı	Sabitlerin tutarlılığını düzeltin
Sayaç geçersiz girdide de artıyor	Doğrulama sayaç artışından sonra yazılmış	Denetimleri sayaç satırının üstüne taşıyın
`uretimSayisi` beklenenden küçük (Görev 5)	Sayaç bildirimi `ORNEK`'ten sonra yazılmış	Sayaç bildirimini yukarı taşıyın
Sabit dizinin içeriği dışarıdan değişiyor	Dizi `public static final` verilmiş	`private` yapıp `clone()` döndüren metot yazın

> **BİLGİ**
>
> Laboratuvar sonunda çözümler paylaşılacaktır. Ancak çözümü görmeden önce görevleri kendiniz tamamlamanız, Hafta 7'deki **miras** konusunu izleyebilmeniz için önemlidir: bu haftanın `private` yapıcı ve `final` sınıf kararları orada doğrudan kalıtım kurallarına bağlanacaktır.